

Chapter 1

Hello, World!

```
1  #include "DwmStreamIO.hh"
2
3  int main(int argc, char *argv[]) {
4      std::string str1("Hello,_world!");
5      std::stringstream ss;
6      if (Dwm::StreamIO::Write(ss, str1)) {
7          std::string str2;
8          if (Dwm::StreamIO::Read(ss, str2)) {
9              std::cout << str2 << '\n';
10         }
11     }
12 }
```

Chapter 2

Supported stream types

2.1 **StreamIO**

`std::istream` and `std::ostream`

2.2 **FileIO**

`FILE *` (`<cstdio>`)

2.3 **DescriptorIO**

UNIX file descriptors

2.4 **BZ2IO**

`BZFILE *`

2.5 **GZIO**

`gzFile`

2.6 **ASIO**

Boost asio sockets

Chapter 3

Directly supported data types

3.1 integral types

3.2 floating point types

3.3 enumerated types

3.4 `std::string`

3.5 `std::vector<bool>`

Chapter 4

Container types

4.1 `std::array<T,N>`

We can stream any instances of `std::array<T,N>` where T is streamable.

4.2 `std::deque<T>`

We can stream any instances of `std::deque<T>` where T is streamable.

4.3 `std::list<T>`

We can stream any instances of `std::list<T>` where T is streamable.

4.4 `std::map<KeyType,MappedType>`

We can stream any instances of `std::map<KeyType,MappedTypeT>` where `KeyType` and `MappedType` are streamable.

4.5 `std::multimap<KeyType,MappedType>`

We can stream any instances of `std::multimap<KeyType,MappedTypeT>` where `KeyType` and `MappedType` are streamable.

4.6 `std::optional<T>`

We can stream any instances of `std::optional<T>` where `T` is streamable.

4.7 `std::pair<FirstType, SecondType>`

4.8 `std::set<T>`

We can stream any instances of `std::set<T>` where `T` is streamable.

4.9 `std::multiset<T>`

We can stream any instances of `std::multiset<T>` where `T` is streamable.

4.10 `std::unique_ptr<T>`

We can stream instances of `std::unique_ptr<T>` where `T` is streamable and the `unique_ptr`'s `deleter_type` is `std::default_delete<T>`. We use the `deleter_type` test to infer that the `std::unique_ptr<T>` points to a single object and not to an array. This can of course be incorrect, and hence should not be relied on as a general mechanism.

4.11 `std::unordered_map<KeyType, MappedType>`

We can stream any instances of `std::unordered_map<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.12 `std::unordered_multimap<KeyType, MappedType>`

We can stream any instances of `std::unordered_multimap<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.13 `std::unordered_set<T>`

We can stream any instances of `std::unordered_set<T>` where `T` is streamable.

4.14 `std::unordered_multiset<T>`

We can stream any instances of `std::unordered_multiset<T>` where `T` is streamable.

4.15 `std::vector<T>`

We can stream any instances of `std::vector<T>` where `T` is streamable.

Chapter 5

Concepts

5.1 Streamable

5.2 C++ iostreams

5.2.1 `HasStreamRead`

5.2.2 `HasStreamWrite`

5.3 `FILE` pointer (`cstdio`)

5.3.1 `HasFileRead`

5.3.2 `HasFileWrite`

5.4 UNIX descriptors

5.4.1 `HasDescriptorRead`

5.4.2 `HasDescriptorWrite`

5.5 Boost asio sockets

5.5.1 `HasAsioRead`

5.5.2 `HasAsioWrite`

5.6 `BZFILE` pointer (`bzip2`)

5.6.1 `HasBZRead`

5.6.2 `HasBZWrite`

5.7 `qzFile` (`libz`)

not we can leverage `StreamIO` to provide `DescriptorIO`, `FileIO`, `BZ2IO`, `GZIO` and `ASIO` functionality automatically (via a pass through a `std::stringstream`).

Chapter 6

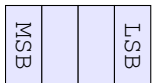
On-the-wire formats

6.1 On-the-wire formats, network byte order (MSB first)

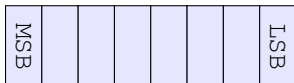
6.1.1 16 bit integral types (`int16_t`, `uint16_t`)



6.1.2 32 bit integral types (`int32_t`, `uint32_t`)

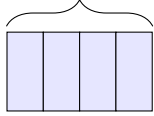


6.1.3 64 bit integral types (`int64_t`, `uint64_t`)



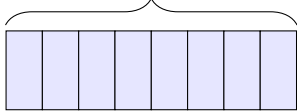
6.1.4 float

4 bytes, RFC4506

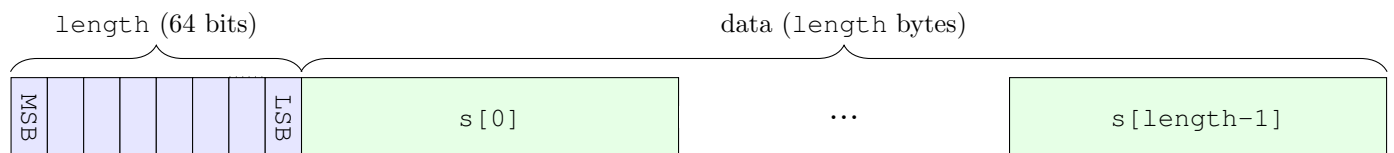


6.1.5 double

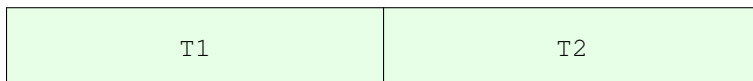
8 bytes, RFC4506



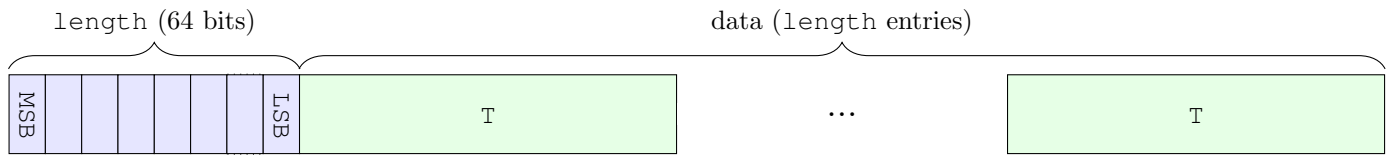
6.1.6 std::string



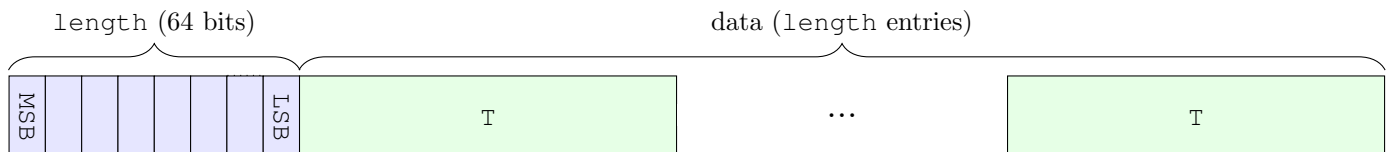
6.1.7 std::pair<T1, T2>



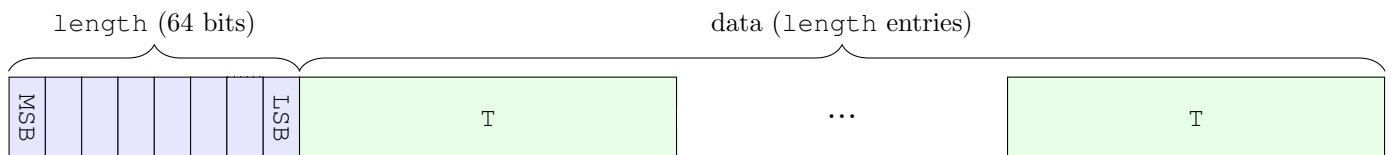
6.1.8 `std::vector<T>`



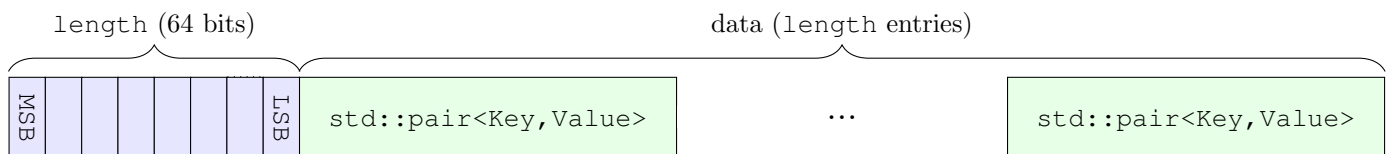
6.1.9 `std::deque<T>`

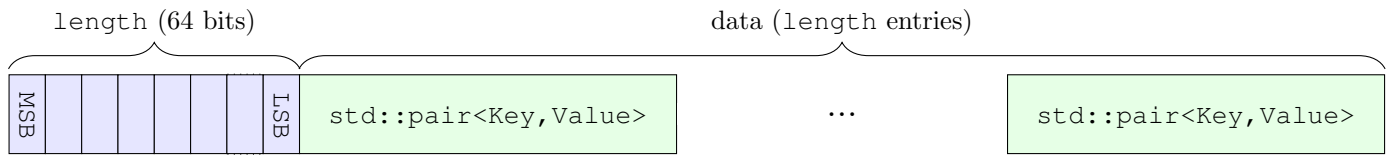
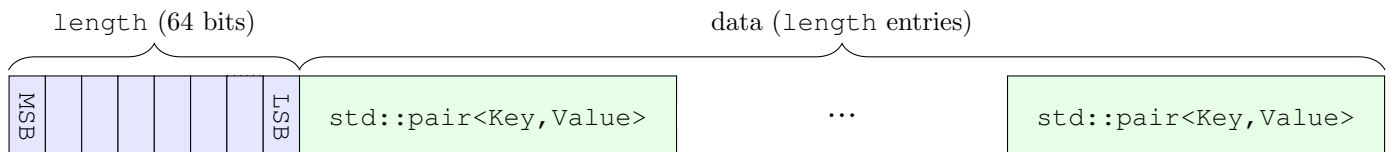
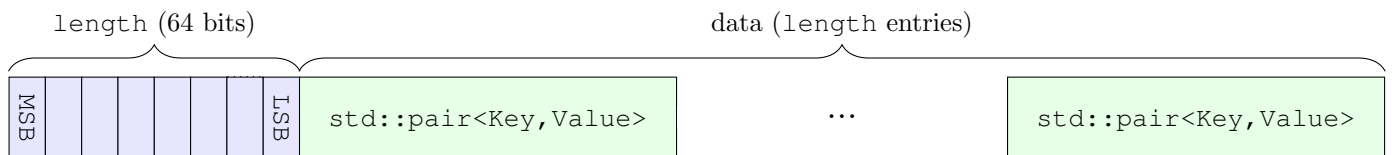
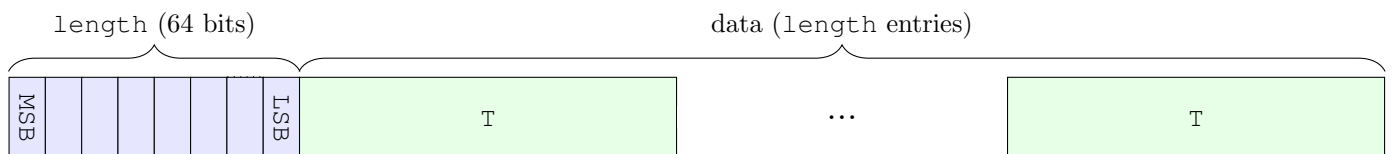


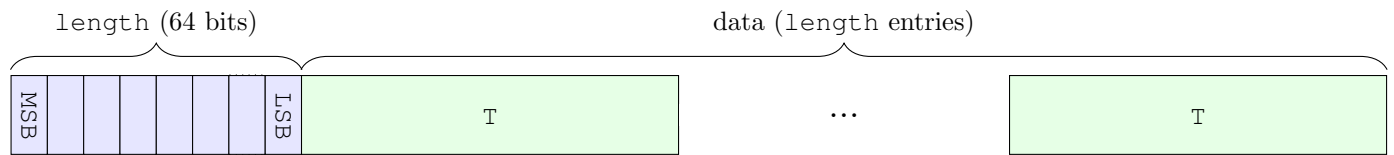
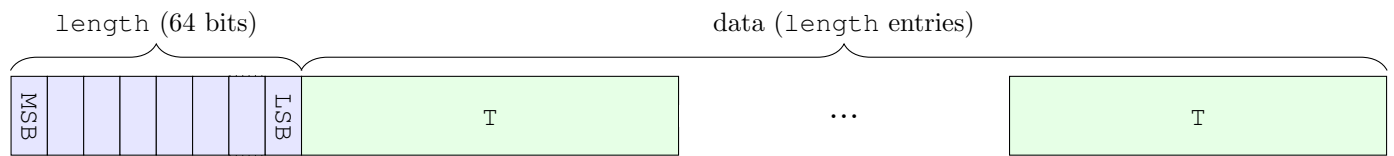
6.1.10 `std::list<T>`



6.1.11 `std::map<Key, Value>`



6.1.12 `std::multimap<Key, Value>`**6.1.13 `std::unordered_map<Key, Value>`****6.1.14 `std::unordered_multimap<Key, Value>`****6.1.15 `std::set<T>`**

6.1.16 `std::multiset<T>`**6.1.17 `std::unordered_set<T>`****6.1.18 `std::unordered_multiset<T>`**